

FUNCTIONAL SPECIFICATION DOCUMENT

Ngekas

Catat pemasukan dan pengeluaran usahamu dengan mudah.

Dari catatan warung di buku tulis, jadi satu aplikasi yang bisa dibuka pemilik & staff dari HP masing-masing.

Daftar Isi

1. Ringkasan Produk
2. Status Saat Ini — Apa yang Sudah Dibangun
3. Prioritas Pengembangan (Roadmap Bertahap)
4. Rincian Fitur per Modul
5. Alur Aplikasi (User Flow)
6. Model Data (Firestore)
7. Kebutuhan Non-Fungsional
8. Keputusan Terbuka & Risiko

Cara Membaca Prioritas

SELESAI

MUST

SHOULD

COULD

NANTI

MUST = wajib ada supaya aplikasi bisa dipakai (MVP). SHOULD = penting tapi bisa menyusul. COULD = nilai tambah kalau ada waktu.
NANTI = sengaja ditunda, butuh keputusan/infrastruktur tambahan.

1. Ringkasan Produk

1.1 Visi

Ngekas adalah aplikasi pencatatan keuangan usaha kecil (warung, toko, jasa) yang menggantikan buku catatan manual. Fokus utamanya: **mencatat pemasukan & pengeluaran secepat dan sesimpel mungkin**, lalu menyajikannya kembali sebagai laporan yang mudah dibaca oleh pemilik usaha — termasuk saat pemilik punya staff yang ikut mencatat transaksi dari HP masing-masing.

1.2 Masalah yang Diselesaikan

Masalah	Solusi Ngekas
Catatan di buku tulis mudah hilang/rusak, tidak ada rekap otomatis	Catatan tersimpan digital, bisa direkap kapan saja
Pemilik tidak tahu untung/rugi tanpa menjumlah manual	Ringkasan & laporan otomatis (harian/bulanan)
Staff mencatat transaksi tapi datanya terpisah dari milik pemilik	Satu workspace bersama — owner & staff mencatat ke data yang sama, dari device berbeda
Staff bisa saja mengubah/menghapus catatan lama secara tidak sengaja	Staff hanya boleh tambah & lihat; ubah/hapus dibatasi untuk owner

1.3 Target Pengguna

Owner (Pemilik Usaha)

Pemilik warung/toko/usaha jasa skala kecil-menengah. Mendaftar sendiri, otomatis menjadi pemilik workspace-nya sendiri. Butuh gambaran cepat kondisi keuangan usaha dan kendali penuh atas data.

Staff (Karyawan/Kasir)

Diundang oleh owner untuk membantu mencatat transaksi harian dari perangkatnya sendiri. Akses dibatasi — bisa mencatat & melihat, tidak bisa mengubah/menghapus data lama atau mengubah pengaturan usaha.

1.4 Platform & Arsitektur Singkat

Aplikasi mobile Flutter (Android prioritas), autentikasi & data bisnis di Firebase (Firebase Auth + Cloud Firestore) supaya data owner dan staff bisa sinkron real-time lintas perangkat.

2. Status Saat Ini — Apa yang Sudah Dibangun

Bagian pondasi (autentikasi & struktur multi-tenant) **sudah berjalan**, dan modul **Kelola Kategori sudah selesai dibangun** (CRUD penuh, real-time via Firestore, dibatasi peran owner/staff). Modul lain (Laporan, Staff, Pengaturan) **belum dibangun** — baru berupa menu placeholder di Home.

Area	Status	Keterangan
Splash & Auto-login	SELESAI	Cek sesi Firebase Auth aktif → langsung ke Home tanpa login ulang
Register	SELESAI	Akun baru otomatis jadi <i>owner</i> dari workspace-nya sendiri (<code>ownerId == uid</code>)
Login / Logout	SELESAI	Termasuk penanganan error via dialog (bukan snackbar)
Profil User (role & ownerId)	SELESAI	Model <code>AppUser</code> , disimpan di Firestore <code>/users/{uid}</code> , dengan cache lokal
Aturan akses data (Firestore Rules)	SELESAI	Baseline: staff boleh create+read, hanya owner boleh update+delete
Home Screen	SEBAGIAN	Kartu profil + grid 4 menu (Laporan, Kelola Kategori, Staff, Pengaturan), routing per-menu lewat <code>MenuItem.id</code> di <code>HomeLogic</code>
Kelola Kategori	SELESAI	Tab Pemasukan/Pengeluaran, tambah/ubah/hapus kategori (ubah & hapus dibatasi owner), list real-time via Firestore stream
Laporan (catat + lihat pemasukan/pengeluaran)	BELUM	Menu ada di Home, halaman & logic belum dibuat
Staff (undang & kelola)	BELUM	Butuh Cloud Function terpisah (client SDK tidak bisa buat akun lain tanpa membajak sesi aktif)
Pengaturan	BELUM	Menu ada di Home, halaman & logic belum dibuat

3. Prioritas Pengembangan (Roadmap Bertahap)

Urutan fase berikut disusun berdasarkan **value ke pengguna paling cepat**: pemilik usaha harus bisa mencatat & melihat hasil catatannya (Fase 1) sebelum fitur kolaborasi tim (Fase 3) jadi relevan. Tidak ada menu "Transaksi" terpisah — mencatat pemasukan/pengeluaran adalah bagian dari menu **Laporan** itu sendiri (satu layar: daftar catatan + tombol tambah), sesuai keputusan produk terbaru.

Fase 1 — MVP: Bisa Catat & Lihat Saldo

MUST

- **SELESAI Kelola Kategori**: buat/lihat/ubah/hapus kategori pemasukan & pengeluaran — dibangun lebih dulu karena Laporan mereferensikan kategori saat mengisi field "tipe"
- **Laporan**: layar utama pencatatan — daftar riwayat pemasukan/pengeluaran + tombol tambah. Mengisi entri = pilih Pemasukan/Pengeluaran → pilih kategori (dari Kelola Kategori, dua langkah) → nominal, tanggal, catatan
- **Ringkasan di Home**: total pemasukan, pengeluaran, dan saldo (hari ini / bulan berjalan)

Fase 2 — Analitik & Kontrol Data

SHOULD

- **Ringkasan periode di Laporan**: filter rentang tanggal, rekap per kategori, grafik pemasukan vs pengeluaran (bagian dari layar Laporan, bukan menu terpisah)
- **Edit/Hapus entri & kategori** (mengacu ke aturan akses yang sudah ada — hanya owner)
- **Pengaturan dasar**: nama usaha, mata uang/format angka, ganti password, logout (sudah ada)

Fase 3 — Kolaborasi Tim (Staff)

COULD

- **Undang Staff**: owner mengundang staff via email (memerlukan Cloud Function)
- **Kelola Staff**: lihat daftar staff, nonaktifkan akses staff
- **UI dibatasi peran**: staff tidak melihat tombol edit/hapus atau menu Pengaturan sensitif

Fase 4 — Penyempurnaan

NANTI

- Export laporan (PDF/Excel), backup/berbagi laporan ke pihak lain (mis. akuntan)
- Target/anggaran bulanan & notifikasi pengingat catat harian
- Keputusan akhir soal `DatabaseService` /sqlite: dijadikan cache offline atau dihapus total
- Mode offline penuh dengan sinkronisasi otomatis saat online kembali

4. Rincian Fitur per Modul

4.1 Autentikasi SELESAI

Fitur	Deskripsi
Daftar Akun	Email + password → otomatis jadi owner workspace sendiri
Masuk	Email + password, error ditampilkan lewat dialog
Sesi Otomatis	Splash screen mengecek sesi aktif, langsung ke Home bila valid
Keluar	Logout dari menu Home, kembali ke Login

4.2 Kelola Kategori SELESAI

Label menu sengaja bukan cuma "Kategori" — kata kerja "Kelola" di depan langsung menandakan ini layar pengaturan/manajemen, bukan sekadar tampilan daftar, supaya lebih jelas buat pengguna awam.

Fitur	Deskripsi
Lihat daftar kategori	Dipisah 2 tab: Pemasukan & Pengeluaran, diperbarui real-time (Firestore stream)
Tambah kategori	Nama + tipe (mengikuti tab aktif saat FAB ditekan). Boleh dilakukan owner & staff
Ubah & hapus kategori	Tombol ubah/hapus hanya tampil untuk owner; ditegakkan juga di <code>firestore.rules</code> (bukan cuma disembunyikan di UI)
Dipakai di mana	Jadi sumber pilihan kategori pada langkah kedua saat mengisi entri di Laporan (lihat Bagian 5.3)

4.3 Laporan (Catat + Lihat) MUST

Tidak ada menu "Transaksi" terpisah — layar ini merangkap pencatatan (tambah entri) dan riwayat/rekap sekaligus.

Fitur	Deskripsi
Tambah entri	Dua langkah: pilih Pemasukan/Pengeluaran dulu → pilih kategori (tersaring sesuai tipe) → isi nominal, tanggal, catatan opsional
Daftar riwayat	Urut dari terbaru, dikelompokkan per tanggal, badge warna beda untuk masuk/keluar
Filter	Berdasarkan tipe dan/atau kategori
Ubah & hapus entri	Hanya owner
Siapa mencatat	Menyimpan siapa (owner/staff) yang menambahkan entri, untuk keterlacakan
Ringkasan periode SHOULD	Total pemasukan/pengeluaran/saldo untuk rentang tanggal, rekap per kategori, grafik tren — bagian lanjutan di layar yang sama, bukan menu baru
Export NANTI	Export ke PDF/Excel — Fase 4

4.4 Staff COULD

Fitur	Deskripsi
Undang staff	Owner memasukkan email staff → sistem membuat akun via Cloud Function dengan <code>role: staff , ownerId: uid-owner</code>
Daftar staff	Owner melihat siapa saja staff yang punya akses ke workspace-nya
Cabut akses	Owner menonaktifkan akun staff tertentu
Batasan tampilan staff	Menu Staff & Pengaturan lanjutan disembunyikan dari akun staff

4.5 Pengaturan SHOULD

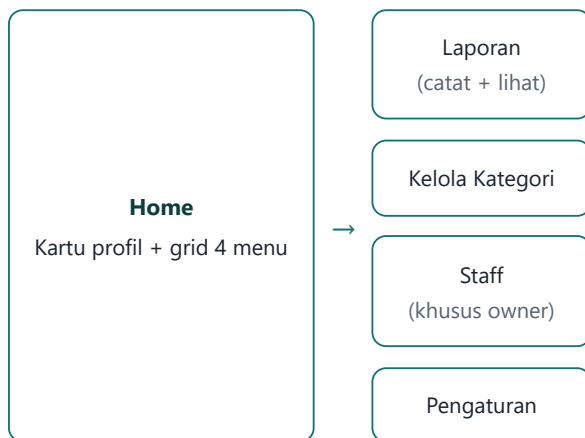
Fitur	Deskripsi
Profil usaha	Nama usaha, opsional logo
Preferensi	Format mata uang/angka
Akun	Ganti password, keluar (sudah ada di Home saat ini, dapat dipindah ke sini)

5. Alur Aplikasi (User Flow)

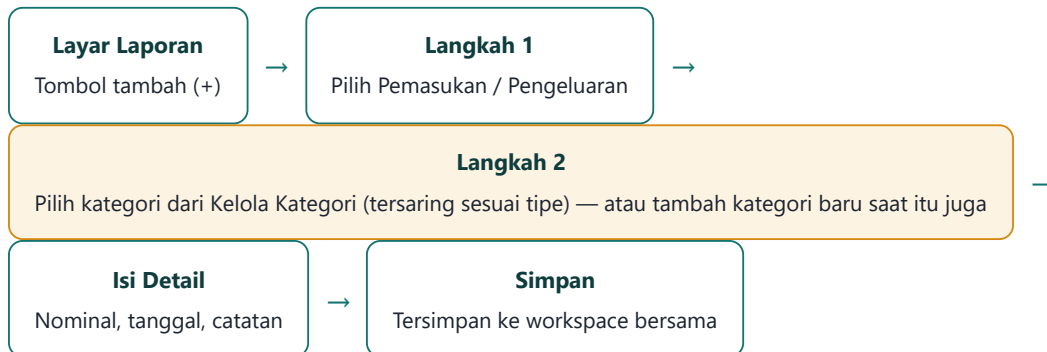
5.1 Alur Buka Aplikasi & Autentikasi



5.2 Alur Navigasi Utama (dari Home)

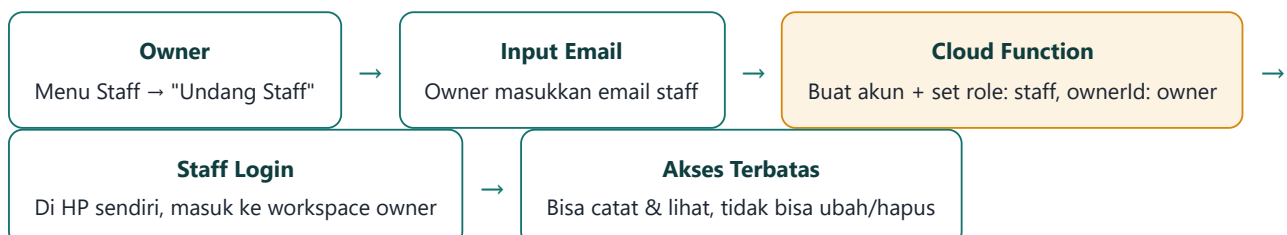


5.3 Alur Isi Laporan — Catat Pemasukan/Pengeluaran (Fase 1 — inti aplikasi)



Field "tipe" pada form Laporan mengambil datanya langsung dari kategori yang sudah dibuat di menu Kelola Kategori (Bagian 4.2) — dua modul ini saling terhubung: Kelola Kategori adalah master data, Laporan adalah tempat memakainya.

5.4 Alur Undang Staff (Fase 3)



Catatan: Alur undang-staff butuh Cloud Function karena Firebase Auth di sisi klien tidak bisa membuat akun baru tanpa membajak sesi login yang sedang aktif (punya owner). Ini alasan fitur Staff ditempatkan di Fase 3, setelah fondasi transaksi/kategori/laporan berjalan.

6. Model Data (Firestore)

Struktur data mengikuti pola *satu workspace per owner*, sehingga owner dan staff yang sama-sama tergabung dalam satu `ownerId` membaca/menulis ke koleksi yang sama.

Koleksi / Path	Isi	Akses
<code>/users/{uid}</code>	Profil akun: <code>email</code> , <code>role</code> (owner/staff), <code>ownerId</code>	Hanya pemilik uid
<code>/workspaces/{ownerId}/categories/{id}</code>	Kategori pemasukan/pengeluaran	Read+create: owner & staff. Update+delete: owner saja
<code>/workspaces/{ownerId}/transactions/{id}</code>	Catatan transaksi (tipe, nominal, kategori, tanggal, catatan, pencatat)	Read+create: owner & staff. Update+delete: owner saja

Aturan akses ini sudah diimplementasikan di `firestore.rules` dan tinggal dipakai saat modul Kategori & Transaksi dibangun — tidak perlu didesain ulang.

7. Kebutuhan Non-Fungsional

Aspek	Kebutuhan
Kecepatan input	Mencatat 1 transaksi harus bisa selesai dalam < 3 tap dari Home
Konsistensi data	Perubahan dari satu device (mis. staff) harus tampak ke device lain (owner) secara near real-time via Firestore
Keamanan & isolasi data	Satu workspace tidak boleh bisa membaca data workspace lain — ditegakkan oleh Firestore Rules, bukan hanya validasi di klien
Skala biaya	Firestore Spark (free tier: 50rb read / 20rb write per hari) sudah dikonfirmasi cukup untuk skala ~50 pengguna aktif
Ketersediaan offline	Belum wajib di Fase 1–2; jadi bahan keputusan di Fase 4 (lihat Bagian 8)
Konsistensi UX error	Semua pesan error/exception ditampilkan lewat <code>AppDialog</code> , bukan <code>SnackBar</code> (standar yang sudah dipakai di modul Auth)

8. Keputusan Terbuka & Risiko

8.1 Nasib `DatabaseService` (sqlite)

Sudah digantikan Firestore sebagai sumber data utama, tapi filenya masih ada dan masih dipanggil sekali saat splash. Perlu diputuskan saat modul Laporan mulai dibangun: dijadikan **cache offline** (dibaca duluan saat tidak ada internet), atau **dihapus total** untuk menyederhanakan codebase.

8.2 Mekanisme Undang Staff

Butuh Cloud Function (server-side) karena Firebase Auth client SDK tidak bisa membuat akun user lain tanpa mengganti sesi login aktif. Ini menambah kebutuhan infrastruktur (Firebase Functions, kemungkinan perlu upgrade dari plan Spark ke Blaze — perlu dicek biayanya sebelum Fase 3 dimulai).

8.3 Granularitas Peran Staff

Saat ini baru ada satu level staff (create+read, tanpa update/delete). Jika ke depan dibutuhkan staff dengan wewenang berbeda (mis. staff senior yang boleh edit), model `role` perlu diperluas dari boolean owner/staff menjadi lebih granular.

8.4 Struktur Menu — Sudah Dikonfirmasi

Revisi dari draf awal dokumen ini: semula ada 5 menu (termasuk "Transaksi" terpisah dari "Laporan"). Pemilik produk mengonfirmasi **tidak perlu menu Transaksi terpisah** — mencatat pemasukan/pengeluaran melebur jadi satu dengan Laporan, dan Kategori berperan sebagai master data yang dipakai lewat proses **dua langkah** saat mengisi entri (pilih Pemasukan/Pengeluaran dulu, baru kategori muncul tersaring). Struktur final: Home → **Laporan, Kelola Kategori, Staff, Pengaturan** (4 menu). Dokumen ini (v1.1) dan implementasi kode sudah disesuaikan dengan keputusan ini.